

API - Studio de cours

Ressources

Cours

- ID (int)
- Nom du cours (string)
- Nom du coach (string)
- Capacité max (int)
- Prix (persisté en int / véhiculé en float)
- Nombre de participants (donnée calculée)

Participants

- ID (int)

Afficher plus ▾

Voici une proposition propre de spécification pour cette première route de l'API.

GET /courses

Retourne la liste de tous les cours disponibles.

Réponse

Status Code : 200 OK

```
[
  {
    "id": 123,
    "name": "Cours de Yoga",
    "coach_name": "Nicolas",
    "seats": 10,
    "price": 19.99,
    "remaining_seats": 9
  },
  {
    "id": 124,
    "name": "Pilates",
    "coach_name": "Marie",
    "seats": 15,
    "price": 24.50,
    "remaining_seats": 12
  }
]
```

JSON



Description des champs

Champ	Type	Description
id	integer	Identifiant du cours
name	string	Nom du cours
coach_name	string	Nom du coach
seats	integer	Capacité maximale
price	float	Prix du cours (stocké en base en entier, exposé en décimal)
remaining_seats	integer	Nombre de places restantes (seats - nombre de participants)

Traitement

Pour chaque cours :

`remaining_seats = seats - nombre_de_participants`



Le prix est :

- stocké en base en **int** (par exemple en centimes)
- renvoyé par l'API en **float**

Exemple :

Prix en base : 1999

Prix API : 19.99



Cette route ne retourne pas la liste des participants, uniquement les informations de chaque cours ainsi que le nombre de places restantes calculé dynamiquement.



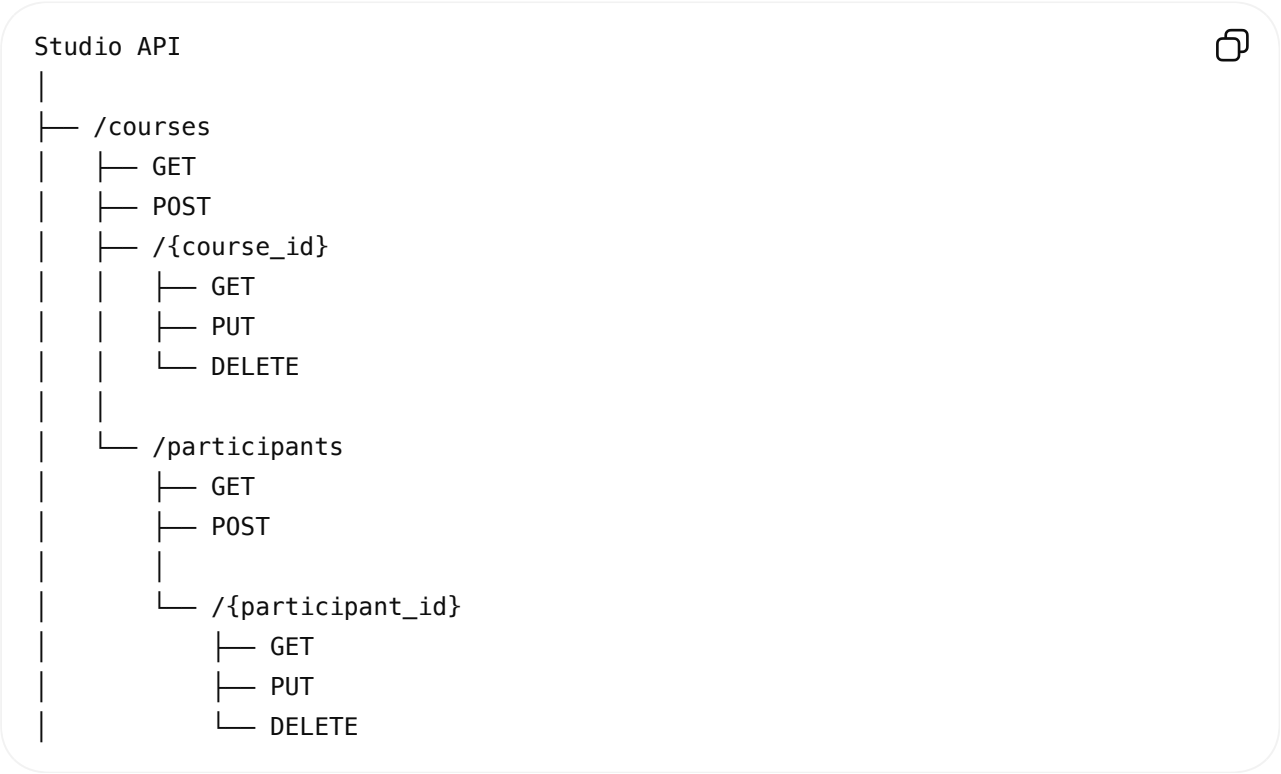
2026-07-17 - Exercice ...
PDF

Evaluation - Cours colle...
PDF

conception de l api

D'après le sujet, tu dois concevoir une API REST qui gère deux ressources : **Courses** et **Participants**.  2026-07-17 - Exercice API - Stu...

Architecture



Ressource Course

Modèle

```
{
  "id": 1,
  "name": "Cours de Yoga",
  "coach_name": "Nicolas",
  "seats": 10,
  "price": 19.99,
}
```

</> JSON 

En base de données

Colonne	Type
id	INTEGER

Colonne	Type
name	VARCHAR
coach_name	VARCHAR
seats	INTEGER
price	INTEGER (centimes)

remaining_seats n'est **pas stocké**, il est calculé :

```
remaining_seats = seats - nombre_de_participants
```



Ressource Participant

```
{
  "id": 15,
  "lastname": "Doe",
  "firstname": "John",
  "age": 35,
  "course_id": 1,
```

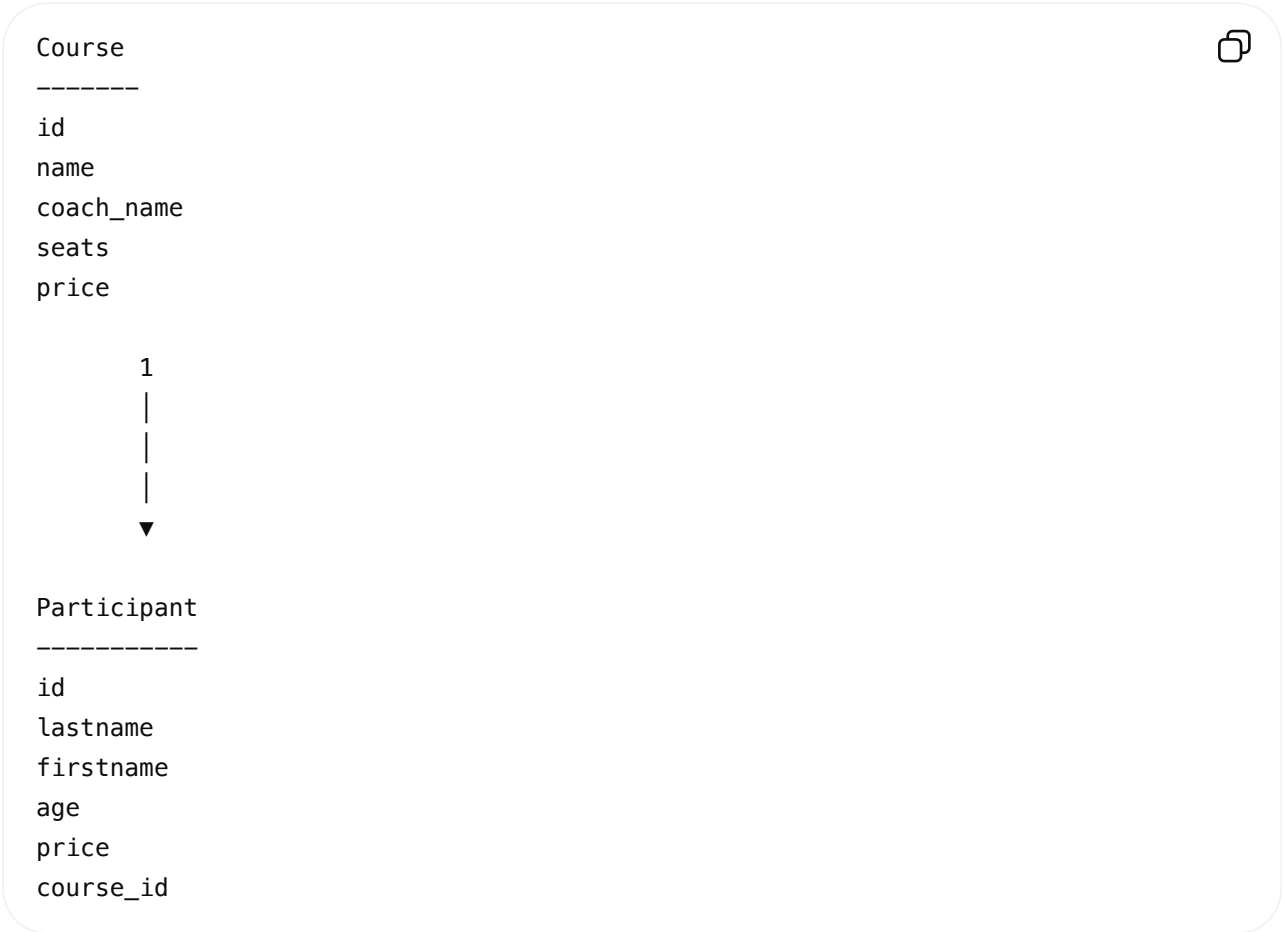
JSON



Base

Colonne	Type
id	INTEGER
lastname	VARCHAR
firstname	VARCHAR
age	INTEGER
course_id	INTEGER (FK)
price	INTEGER (centimes)

Relation



Un cours possède **plusieurs participants**.

Un participant appartient à **un seul cours**.

Relation **One-To-Many**.

Endpoints

Courses

Méthode	URI	Description
GET	/courses	Liste des cours
GET	/courses/{id}	Détail d'un cours
POST	/courses	Création
PUT	/courses/{id}	Modification
DELETE	/courses/{id}	Suppression

Méthode	URI	Description
GET	/courses/{id}/participants	Liste des participants d'un cours
GET	/courses/{id}/participants/{p_id}	Un participant
POST	/courses/{id}/participants	Inscrire un participant
PUT	/courses/{id}/participants/{p_id}	Modifier une inscription
DELETE	/courses/{id}/participants/{p_id}	Désinscrire un participant

2026-07-17 - Exercice API - Stu...

```
graph TD
    subgraph courses
        id1[id PK]
        name1[name]
        coach_name[coach_name]
        seats1[seats]
        price1[price int]
    end
    subgraph instructors
        id2[id PK]
        lastname[lastname]
        firstname[firstname]
        age[age]
        price2[price int]
        course_id[course_id FK]
    end
    coach_name ---|1 N| id2
```

Organisation du projet (Flask)

```
studio_api/
├── app.py
├── config.py
├── database.py
├── models/
│   ├── course.py
│   └── participant.py
├── routes/
│   ├── courses.py
│   └── participants.py
├── services/
│   ├── course_service.py
│   └── participant_service.py
├── schemas/
│   └── course_schema.py
└── requirements.txt
```

[Demander à ChatGPT](#)[Commencer à écrire](#)

Cette conception respecte les principes REST, sépare clairement les ressources **Course** et **Participant**, utilise une relation **1-N** entre elles, et correspond aux routes et aux données demandées dans l'énoncé.

       Sources